JC2001: Introduction to Software Engineering

Lecture 2: Software processes & Agile software development

Lecture Schedule: Week 2, Day 1

Dr Zhenhua Yu zhenhua.yu@abdn.ac.uk

Dr Riazul Islam riazul.islam@abdn.ac.uk

Dr Shahzad Mumtaz shahzad.mumtaz@abdn.ac.uk

Course Delivery Team

Dr Zhenhua Yu

zhenhua.yu@abdn.ac.uk



Dr Riazul Islam

riazul.islam@abdn.ac.uk



Dr Shahzad Mumtaz

shahzad.mumtaz@abdn.ac.uk



Outline

- Software Processes (Chapter 2)
- Agile software development (Chapter 3)

Software Processes
(Software Development Paradigms)



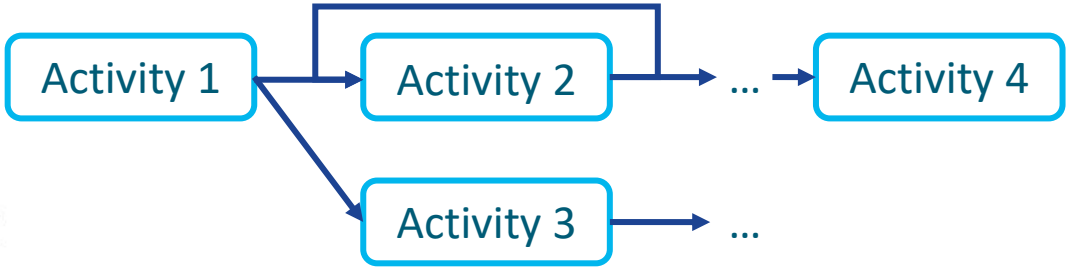
1495 UNIVERSITY OF ABERDEEN

Software Processes

- A software process is a set of related activities that leads to the production of a software system.
- (There is no universally applicable software process.)



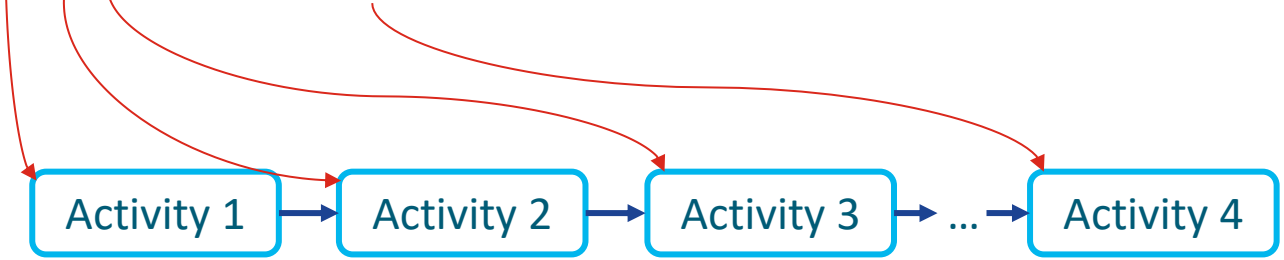
-- E.g. A process



-- E.g. Another process

Software Processes

- Software processes must include the four fundamental software engineering activities:
  1. Software specification – The functionality of the software and constraints on its operation must be defined.
  2. Software development – The software to meet the specification must be produced.
  3. Software validation – The software must be validated to ensure that it does what the customer wants.
  4. Software evolution – The software must evolve to meet changing customer needs.



Software Processes (activities)

1. **Software specification**
    - It is to understand and define what services are required from the system and identifying the constraints on the system's operation and development.
    - Requirements engineering is a particularly critical stage of the software process, as mistakes made at this stage inevitably lead to later problems in the system design and implementation.
    - There are three main activities in the requirements engineering process
        - Requirements elicitation and analysis
        - Requirements specification
        - Requirements validation
    - Outputs of this activity is requirements document.
    - (We will go into detail in later lessons)

Software Processes (activities)

2. Software design and implementation

- Software design and implementation can be one activity or two separated activities.
- **Design** – the activities in the design process vary, depending on the type of system being developed. It usually includes architectural design, database design, interface design, component selection and design, etc. The corresponding outputs are the documents – system architecture, database design, interface specification, component descriptions.
- **Implementation** (programming) – is an individual activity, and there is no general process that is usually followed.
- (We will go into detail in later lessons)

Software Processes (activities)

3. Software validation

- Software validation or, more generally, verification and validation (V & V) is intended to show that a system both conforms to its specification and meets the expectations of the system customer.
- Program testing, where the system is executed using simulated test data, is the principal validation technique.
- Validation may also involve checking processes, such as inspections and reviews, at each stage of the software process from user requirements definition to program development.
- (Most V & V time and effort is spent on program testing.)
- (We will go into detail in later lessons)

Software Processes (activities)

4. Software evolution

- Large software systems usually have a long lifetime. During their lifetime, operational software systems have to change if they are to remain useful.
- The aim of software evolution is to add, change, or remove features in an existing software. It can be regarded as a maintenance process or a new development process.
- There is no such thing as a standard software change or evolution process.
- (We will go into detail in later lessons)

Mock quiz

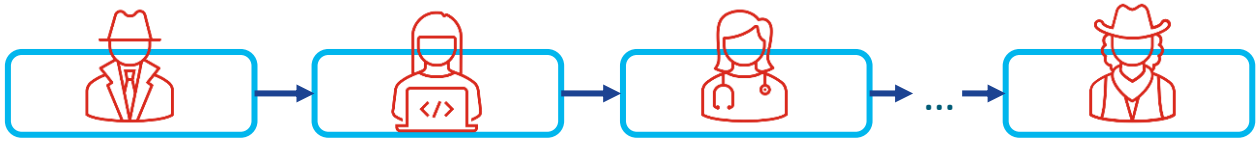
- Which one of the following is *not* a software process activity?
  - Software evolution
  - Software validation
  - Program testing
  - Software specification

Software Processes

- When describing processes, it is also important to describe who is involved, what is produced, and conditions that influence the sequence of activities.

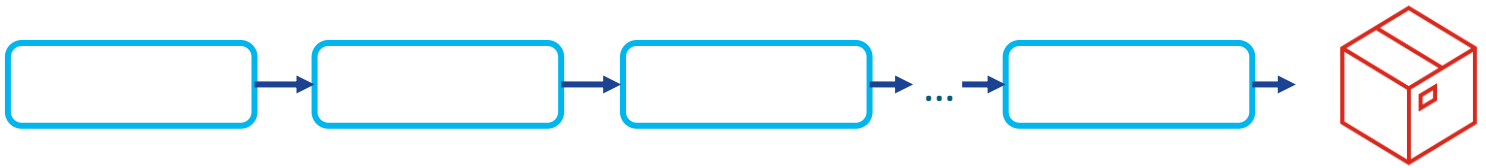
Software Processes

- When describing processes, it is also important to describe who is involved, what is produced, and conditions that influence the sequence of activities.
    - Roles reflect the responsibilities of the people involved in the process. Examples of roles are project manager, configuration manager, and programmer.



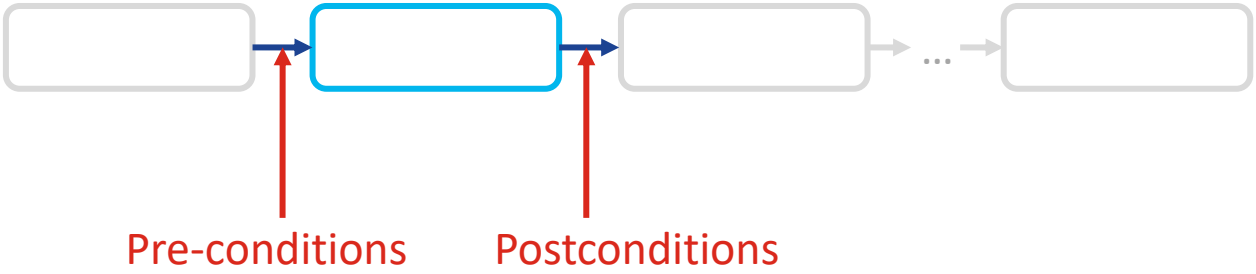
Software Processes

- When describing processes, it is also important to describe who is involved, what is produced, and conditions that influence the sequence of activities.
    - Products or deliverables are the outcomes of a process activity. For example, the outcome of the activity of architectural design may be a model of the software architecture.



Software Processes

- When describing processes, it is also important to describe who is involved, what is produced, and conditions that influence the sequence of activities.
    - Pre- and postconditions are conditions that must hold before and after a process activity has been enacted or a product produced. For example, before architectural design begins, a precondition may be that the consumer has approved all requirements; after this activity is finished, a postcondition might be that the UML models describing the architecture have been reviewed.

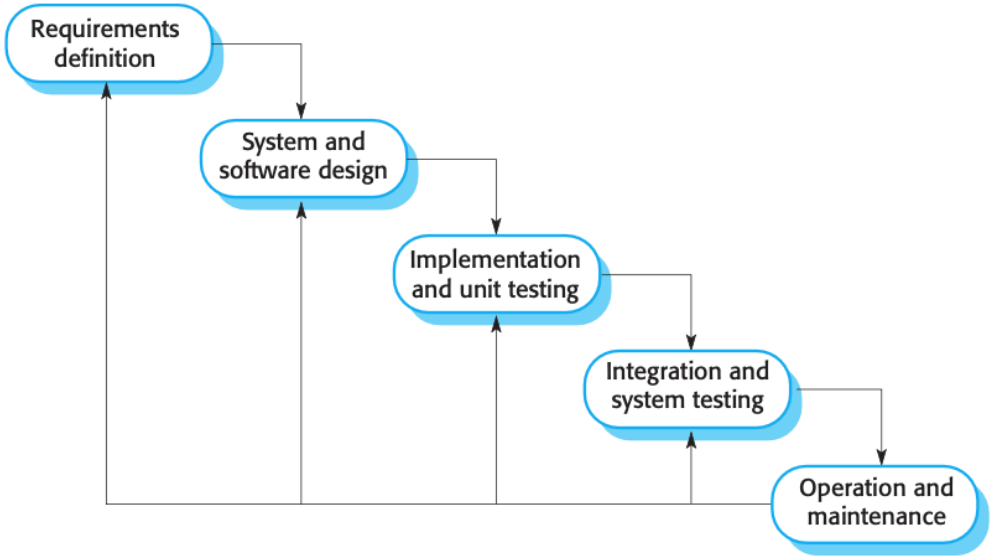


Software Process Models

- A software process model (sometimes called a Software Development Life Cycle or SDLC model) is a simplified representation of a software process.
- General process models (sometimes called *process paradigms*) include:
    - Waterfall model
    - Incremental development
    - Integration and configuration
    - Agile process (in next Chapter)
    - …

Waterfall Model

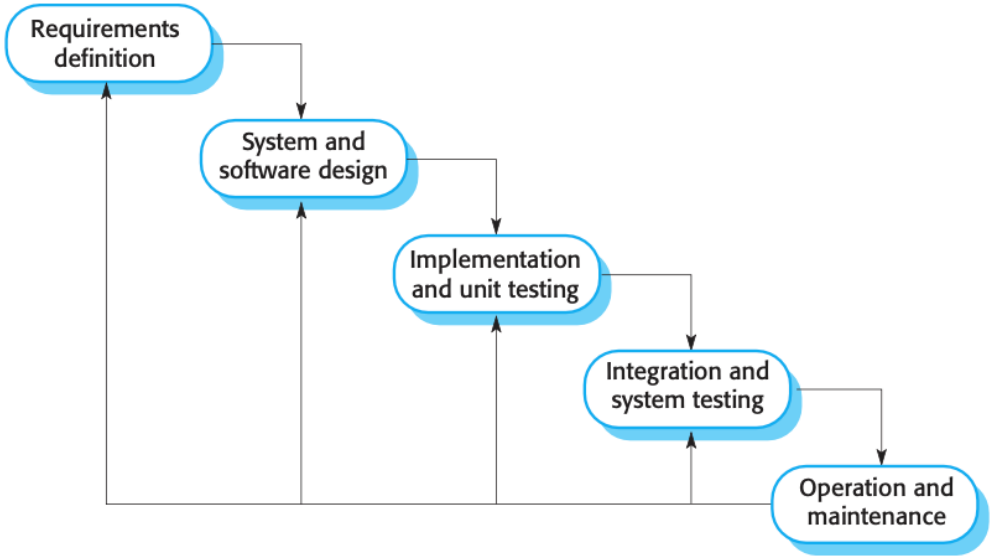
- It presents the software development process as a number of stages (activities)



Waterfall Model

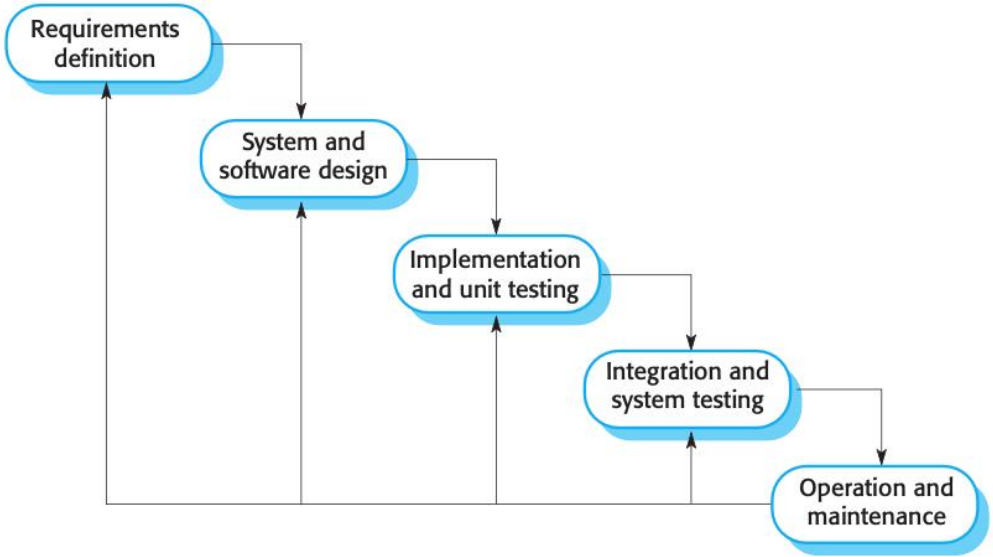
Pre- and postcondition

- The result of each stage is one or more documents that are approved ("signed off"). The following stage should not start until the previous phase has finished.



Waterfall Model

- The waterfall model is a plan-driven process – you plan and schedule all of the process activities before starting software development.
- When you encounter a problem at a certain stage, you solve the problem under the current stage. You cannot go back to the previous stage (otherwise you cannot control risks).



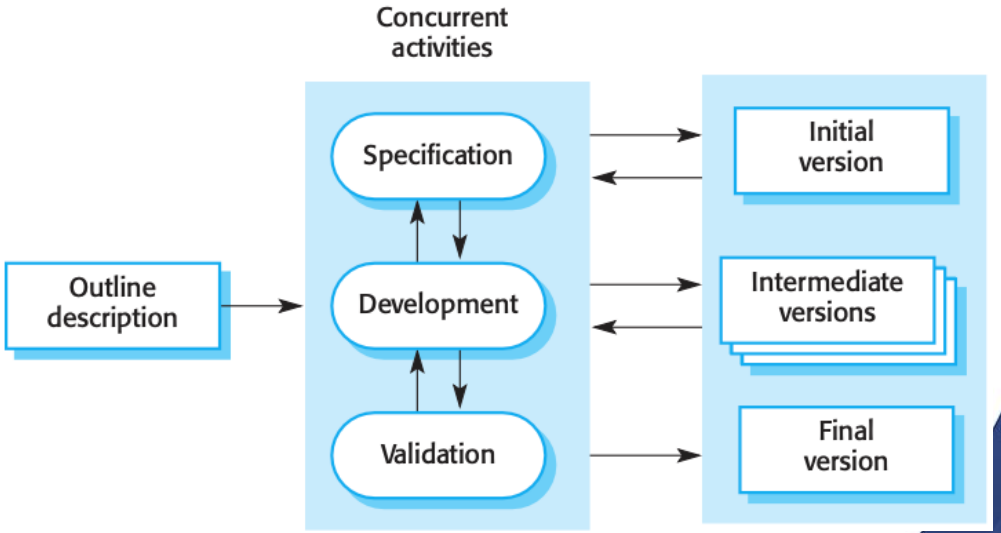
Waterfall Model

So, the waterfall model is only appropriate for some types of target software products:

- The product is very simple or small, so that you can definitely figure out all the potential issues in each stage.
- All the potential issues can be predicted in each stage. E.g. Critical systems require extensive safety and security analysis (safety-related problems in the specification and design are usually very expensive to correct at the implementation stage), so we may assume we can find all the potential issues.
- The development can only use waterfall model because of some external reasons. E.g. Large software systems that are part of broader engineering systems developed by several partner companies. The communication between each companies may be very expensive.

Incremental Development

- Incremental development is based on the idea of developing an initial implementation, getting feedback from users and others, and evolving the software through several versions until the required system has been developed.
- Specification, development, and validation activities are interleaved rather than separate, with rapid feedback across activities.



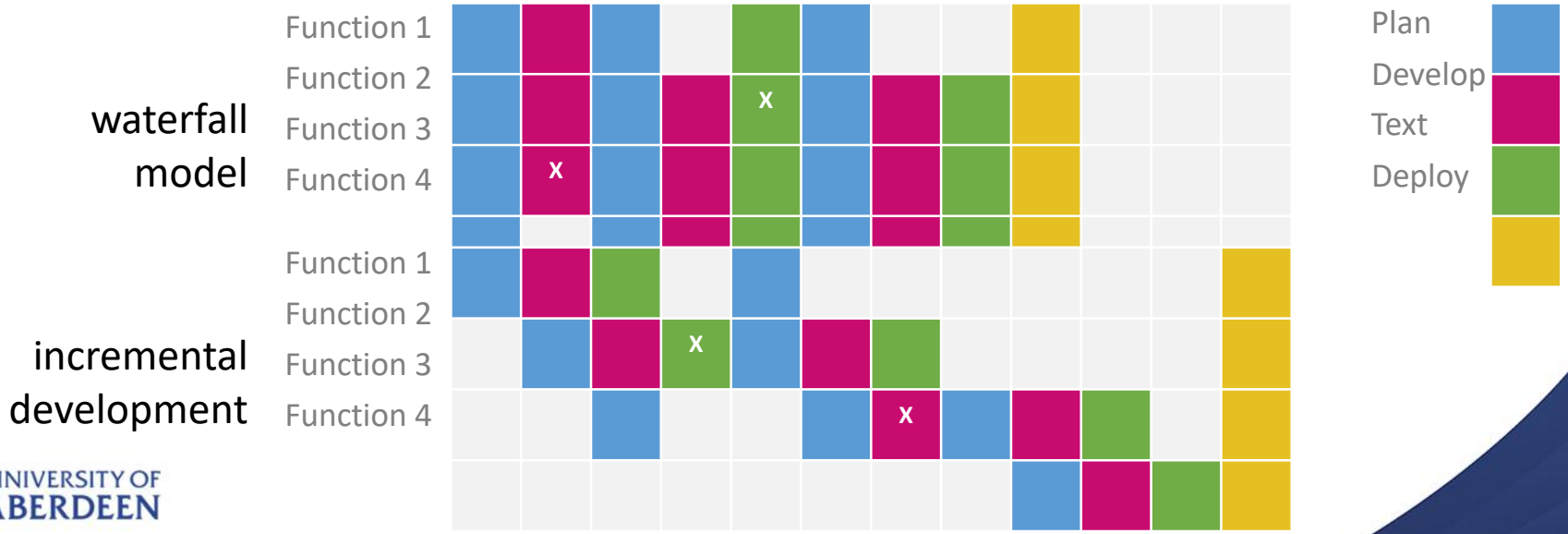
Incremental Development (advantages)

Incremental development has three major advantages over the waterfall model:

1. The cost of implementing requirements changes is reduced.
2. It is easier to get customer feedback
3. Customers are able to use and gain value from the software earlier than a waterfall process.

Incremental Development (advantage 1)

1. The cost of implementing requirements changes is reduced.
   (The amount of analysis and documentation is significantly less than is required with the waterfall model.)



Incremental Development (advantage 2)

2. It is easier to get customer feedback on the development work that has been done. Consumers can participate in software development, so they can
    - find problem on very early stages.
    - find it difficult to judge progress from software design documents.
    - give up some requirements which are difficult to implement

Incremental Development (advantage 3)

3. Customers are able to use and gain value from the software earlier than a waterfall process.
    - Release software as fast as possible
    - Easier to capture the market
    - Profit as quickly as possible

Incremental Development (limitation)

From a management perspective, the incremental approach also has problems:

- The process is not visible. Managers need regular deliverables to measure progress.
- If systems are developed quickly, it is not cost effective to produce documents that reflect every version of the system.
- System structure tends to degrade as new increments are added. Regular change leads to messy code as new functionality is added in whatever way is possible. It becomes increasingly difficult and costly to add new features to a system.
- Not suitable for large, complex, long-lifetime systems. Large systems need a stable framework or architecture, and different teams need to know their works at the very beginning. This has to be planned in advance rather than developed incrementally.

Mock quiz

- You are designing the cooling system for a new nuclear power plant. Which of the following software process model would you use: (a) waterfall model, or (b) incremental development model? Why did you choose one model over the other model?

Integration and Configuration

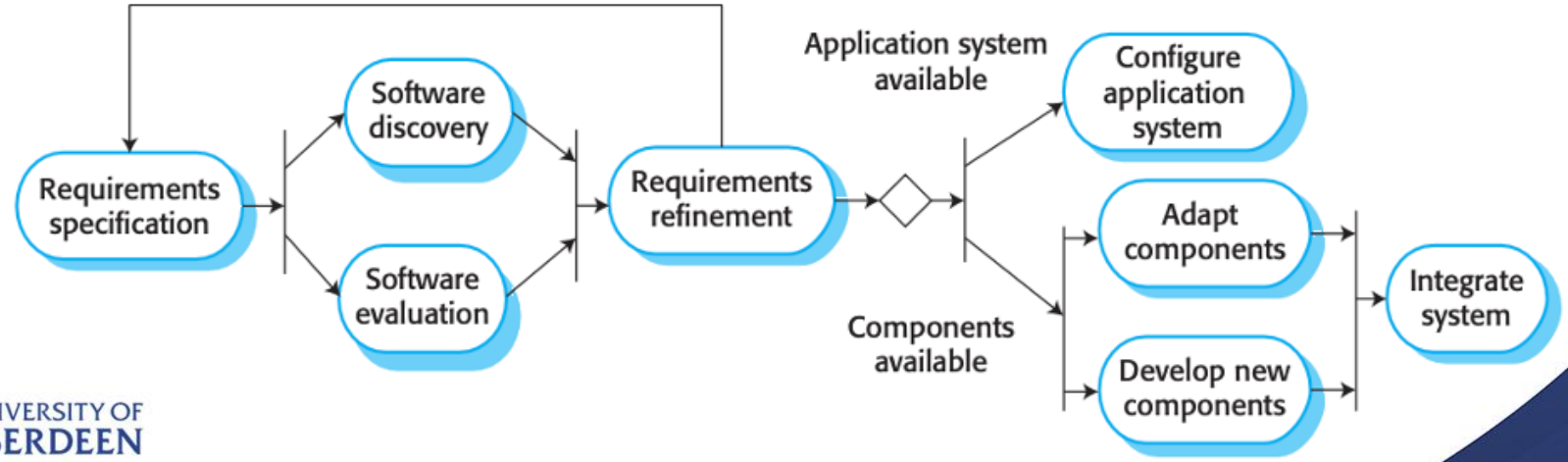
Integration & Configuration focus on reuse the previously developed programs.

Three types of software components are frequently reused:

- Stand-alone application systems that are configured for use in a particular environment. These systems are general-purpose systems that have many features, but they have to be adapted for use in a specific application.
- Collections of objects that are developed as a component or as a package to be integrated with a component framework (e.g. third party library in Python).
- Web services (e.g. HTTP API) that are developed according to service standards and that are available for remote invocation over the Internet.
- …

Integration and Configuration

- So, the process of Integration & Configuration focus on how to reuse programs to implement new function instead of develop new components.
- We will explain this topic in detail in later lectures (in Chapter 15 – 18).



Integration and Configuration

Advantages:

- Significantly reduces software development workload (reduce costs, increase speed)
- Significantly reduce testing workload and easily ensure product quality

Disadvantages:

- Some requirements cannot be achieved (limited by the reusable components).
- May lost the control of the system evolution as new versions of the reusable components are not under the control.

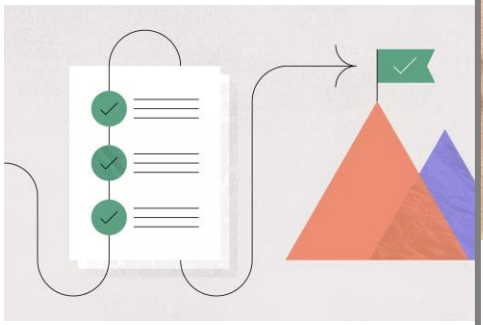
Agile Software Development



1495 UNIVERSITY OF ABERDEEN

Carefully Plan vs. **Respond to Changes** (agile)

- 1980 – 1990
- large teams
- geographically dispersed
- long time periods



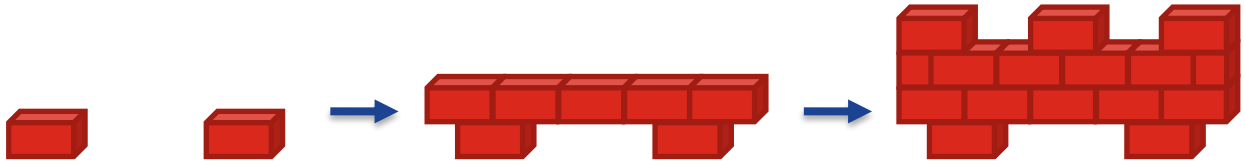
Carefully Plan

Respond to Changes

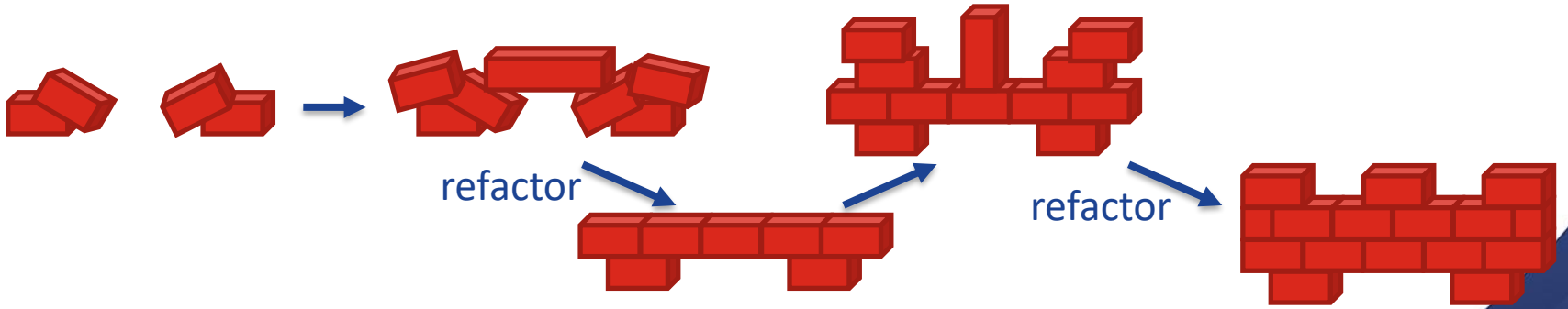


- after 1990
- quick & low cost
- rapidly react to changes

Carefully Plan vs. Respond to Changes



A typical carefully-planned development



A typical agile development

Manifesto for Agile Software Development

“We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.”

The Principles of Agile Methods

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
Embrace change	Expect the system requirements to change, and so design the system to accommodate these changes.
Incremental delivery	The software is developed in increments, with the customer specifying the requirements to be included in each increment.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.
People, not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.

Agile Software Development

- There are many Agile methods/tools, e.g. Extreme Programming, Scrum, FDD, DSDM, Crystal, ASD. (A development process can choose one or more of them to use.)
- All agile methods share some common characteristics:
  - Agile methods are designed to produce useful software quickly.
  - Agile methods are kinds of Incremental Development methods, but usually use small increments. Typically, new releases of the system are created and made available to customers every two or three weeks.
  - The processes of specification, design and implementation are interleaved.
  - Extensive tool support is used to support the development process.

Agile Software Development

Extreme Programming



Extreme Programming

- Extreme programming (XP) is a common used agile development method.
- Extreme programming introduced a number of agile practices that were quite different from the development practice of that time.

  - Collective ownership
  - Continuous integration
  - Incremental planning (story cards)
  - On-site customer
  - Pair programming
  - Refactoring
  - Simple design
  - Small releases
  - Sustainable pace
  - Test first development

Extreme Programming

User stories

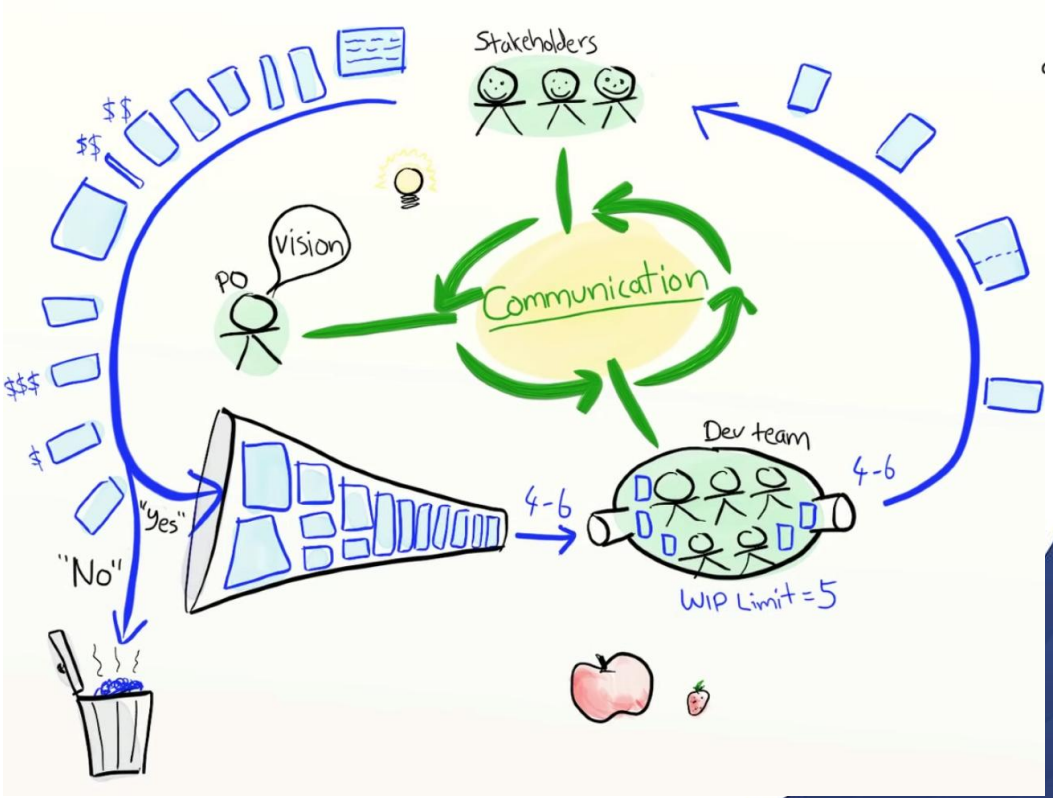
- User cards are the requirement specification of XP, which are created by the consumers and development team together.
- User stories can be helpful in getting users involved in suggesting requirements.
- A user card example (see next page)
- Each user card refers to a system function. (surely, if consumers proposed too many user cards, some of them may need to be discards.)

STORY CARD NO: 16	Project Name E-Commerce	Estimation: 4 Hours
Story Name: User Registration		Date: 16/08/2007 1:30 PM
STORY: User needs to register with unique username and password before purchasing anything from the online store		Acceptance Test: 1. User Id must be unique 2. Try to register with duplicate user id and Password 3. Try to register user name only 4. Try to register with password only 5. Forget Password Link
Note: User Can View or Visit store as a Visitor but needs to register before purchasing anything		Risk: Low

An example story card describing a user story for user registration

Development Process with Story Cards

- Consumers create story cards.
- Dev team releases stories (i.e. the updated system version) early and often.



Extreme Programming

Refactoring

Examples of refactoring include the reorganization of a class hierarchy to remove duplicate code, the renaming of attributes and methods, and the replacement of similar code sections, with calls to methods defined in a program library.

- Traditional software engineering
  - design for change – anticipate future changes to the software and design it so that these changes can be easily implemented
- Extreme programming
  - Never design for change
  - Use frequent refactoring to tidy code

Extreme Programming

Test-first development (test driven development)

- Test-driven is one of the most important innovations in software engineering.
- Traditional software programming
    1. Analyse the business logic, then, implement the function for the logic
    2. Create test cases to test the function
- Extreme programming
    1. Create test cases
    2. Write code only to let the function pass the test; just ignore the business logic!

Extreme Programming

Pair programming

- Pair programming is another one of the most important innovations.
- Two programmers code simultaneously, but use one computer with one keyboard.
- In each pair, one programmer (navigator) tells the other programmer (driver) what to code, and the driver use the keyboard to program.


✓


X


X


X

Extreme Programming

Pair programming

- It will help greatly in improving software quality.
  - Because the programming involves communication, the two programmers are easy to find each other's problems to avoid potential bugs.
  - The navigators tend to think with user's perspective, so the design of the code can be therefore improved.
- It can help new employees get started quickly.

Extreme Programming

- Extreme programming (XP) is a common used agile development method.
- Extreme programming introduced a number of agile practices that were quite different from the development practice of that time.

  - Collective ownership
  - Continuous integration
  - Incremental planning (story cards)
  - On-site customer
  - Pair programming
  - Refactoring
  - Simple design
  - Small releases
  - Sustainable pace
  - Test first development

Agile Software Development

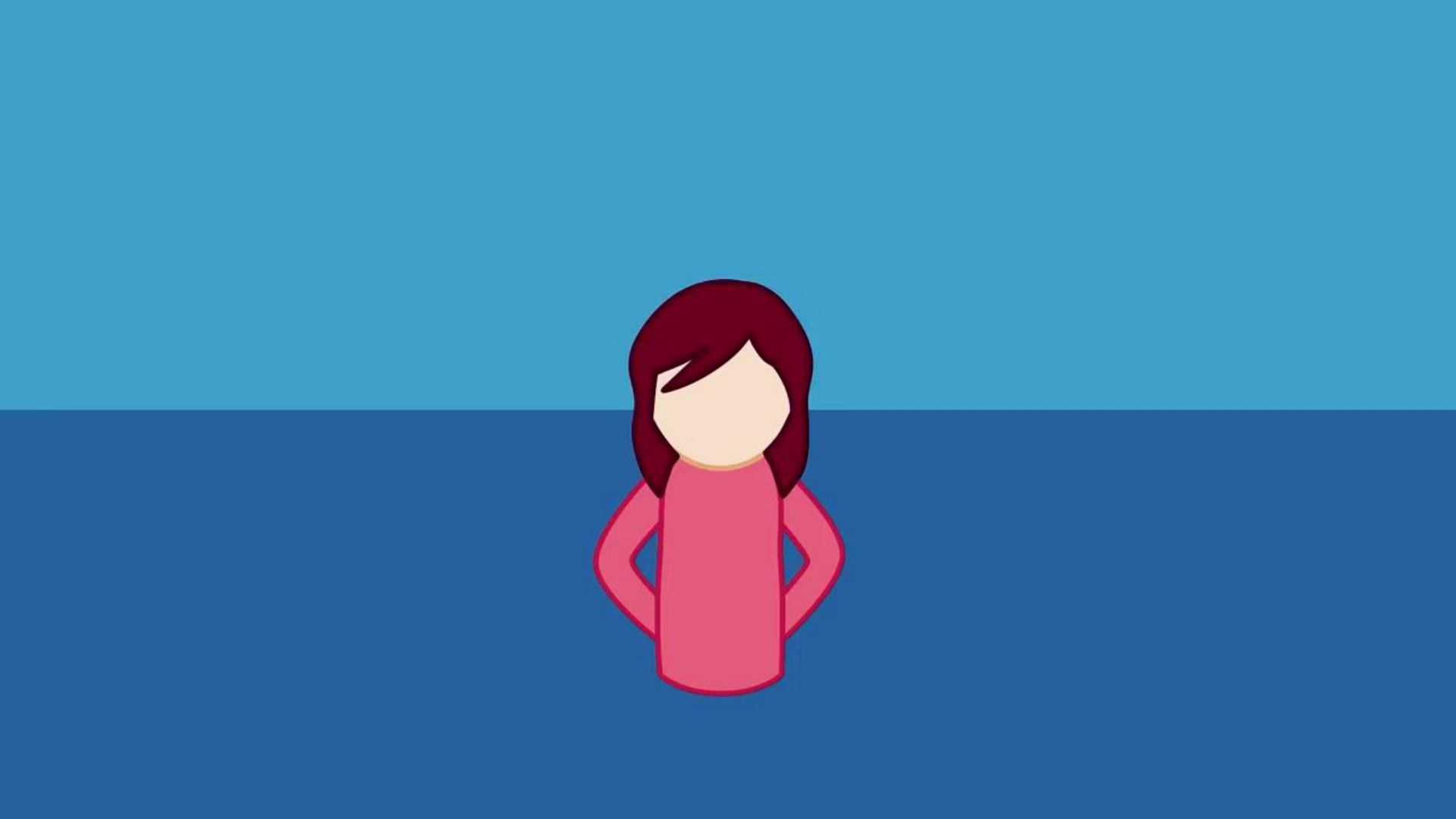
Scrum



1495 UNIVERSITY OF ABERDEEN

Scrum

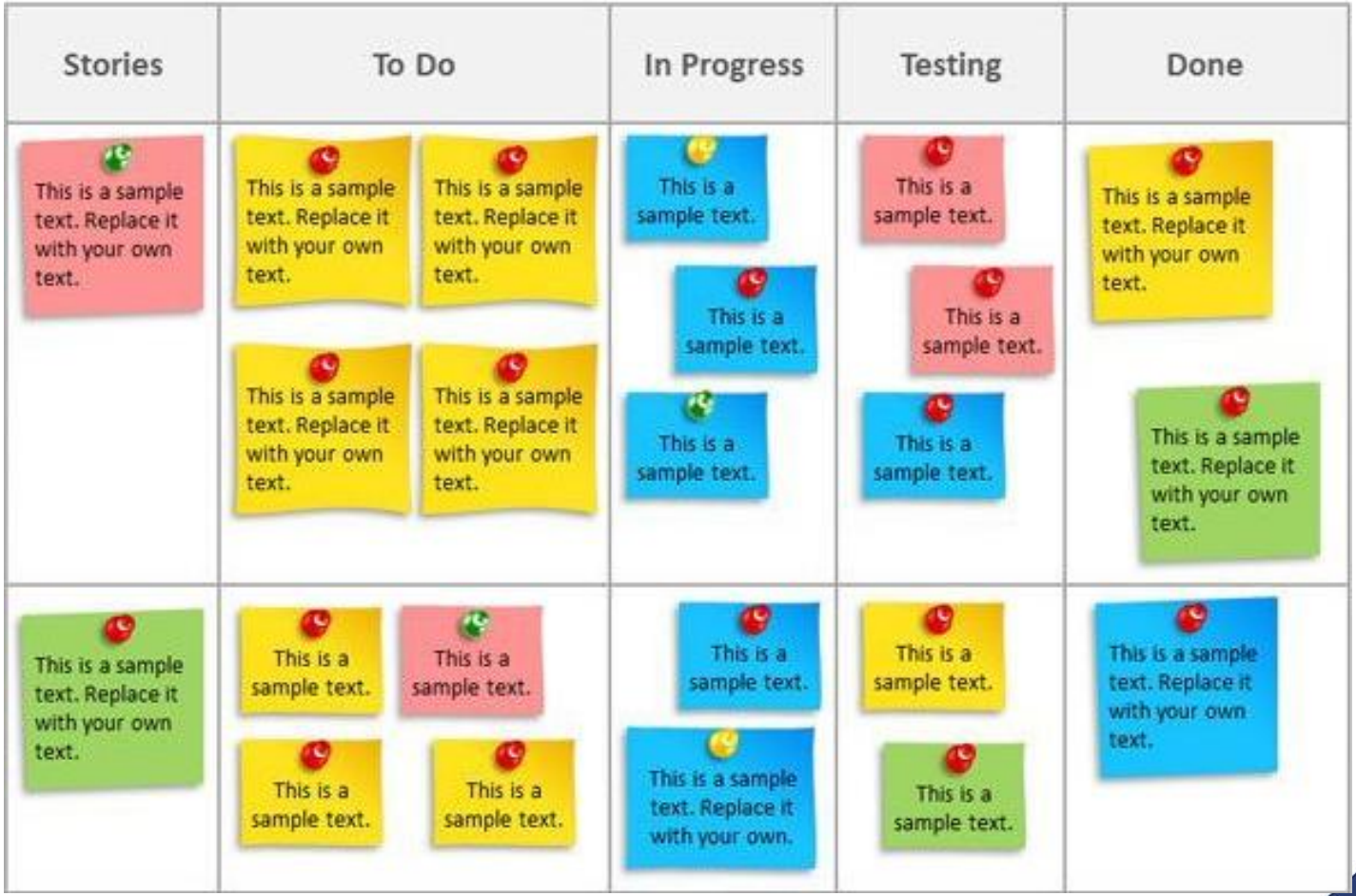
- Scrum is a method/framework to manage an agile project.
- We use an video to introduce Scrum…



Scrum

Key components

- 3 Roles -- Product owner, ScrumMaster, Development team
- StandUp Meeting -- A daily meeting of the Scrum team that reviews progress and prioritizes work to be done that day. Ideally, this should be a short face-to-face meeting that includes the whole team.
- Sprint -- A development iteration. Sprints are usually 2 to 4 weeks long.
- Scrum Task Board --



A Scrum Task Board example used by a Scrum team


Daily StandUp Meeting

Summary

- What are the software processes.
  - A software process consists with activities.
  - Waterfall model
  - Incremental development
  - Integration and configuration
- Agile software development
  - Agile software development is incremental development
  - What is Extreme Programming – an agile programming method
  - What is Scrum – an agile management method